

FINAL PROJECT

Course: Introduction To Artificial Intelligence

Nguyen Nhut Khang
Student

*Faculty of Information Technology
Ton Duc Thang University
Ho Chi Minh city, Vietnam
523H0145@student.tdtu.edu.vn*

Nguyen Tran Hoang Nhan
Student

*Faculty of Information Technology
Ton Duc Thang University
Ho Chi Minh city, Vietnam
523H0164@student.tdtu.edu.vn*

Phan Thanh Dat
Student

*Faculty of Information Technology
Ton Duc Thang University
Ho Chi Minh city, Vietnam
523H0124@student.tdtu.edu.vn*

Nguyen Vo Thanh Tu
Student

*Faculty of Information Technology
Ton Duc Thang University
Ho Chi Minh city, Vietnam
523H0189@student.tdtu.edu.vn*

Mai Hoang Thai
Student

*Faculty of Information Technology
Ton Duc Thang University
Ho Chi Minh city, Vietnam
523H0177@student.tdtu.edu.vn*

Nguyen Thanh An
Lecturer

*Faculty of Information Technology
Ton Duc Thang University
Ho Chi Minh city, Vietnam
nguyenthanhan@tdtu.edu.vn*

Abstract—This report details the practical application of fundamental Artificial Intelligence techniques across four distinct computational tasks, demonstrating a comprehensive understanding of diverse AI paradigms. Key applications involve optimizing 3D surfaces with Simulated Annealing, strategic game playing using Heuristic Alpha-Beta search, solving logic-based constraint satisfaction problems via SAT solvers, and building predictive models with Naïve Bayesian classification. For each task, this report summarizes the specific requirements, methodologies employed, key experimental results (illustrated with figures/tables where applicable), and overall completion levels. This collective work showcases the effective application of these core AI algorithms to complex problem-solving.

Index Terms—Artificial Intelligence, Simulated Annealing, Alpha-Beta Pruning, Heuristic Search, Constraint Satisfaction Problems, SAT Solvers, Naïve Bayes Classifier, Propositional Logic.

I. INTRODUCTION

This report details the methodologies, implementations, and outcomes of applying several core Artificial Intelligence techniques to a diverse set of computational problems. We begin by reviewing the essential AI concepts that form the theoretical basis for our work. Subsequently, we present the specific approaches engineered for each of the four tasks:

- Optimizing complex multivariable functions, such as 3D surfaces, using Simulated Annealing (SA), which included the design of custom cooling schedules to guide the search.
- Creating intelligent game-playing agents for a 9×9 Tic-Tac-Toe variant (requiring four pieces in a row to win) using heuristic Alpha-Beta search, a process that involved developing appropriate evaluation functions and determining effective search depth limits.

- Solving intricate constraint-based puzzles, exemplified by grid matrix coloring based on adjacent cell counts, by translating these problems into propositional logic and employing SAT solvers like Glucose3 to find valid solutions.
- Building predictive models for tasks such as student academic outcome prediction from continuous data (quiz scores); this required the manual implementation of Naïve Bayes classifiers coupled with suitable data discretization techniques to handle the continuous input.

II. METHODOLOGIES AND SYSTEM IMPLEMENTATION

A. Task 1: Simulated Annealing Search on 3D Surfaces

1) **Issue:** The objective is to find the global maximum of the function $f(x, y) = \sin\left(\frac{x}{8}\right) + \cos\left(\frac{y}{4}\right) - \sin\left(\frac{x \cdot y}{16}\right) + \cos\left(\frac{x^2}{16}\right) + \sin\left(\frac{y^2}{8}\right)$. The search starts from the origin $O(0, 0)$. Actions are constrained to step sizes of 0 or $\pm \frac{\pi}{32}$ in both x and y directions. The function's complexity with numerous local maxima necessitates an algorithm that can avoid getting trapped.

2) **Approach and Formulate:** Simulated Annealing (SA) was employed due to its effectiveness in escaping local optima in complex search spaces. The problem formulation for SA includes:

- **State:** Coordinates (x, y) on a 2D plane.
- **Objective Function:** The goal is to maximize the given $f(x, y)$.
- **Actions (Neighborhood Generation):** Successor states are generated by applying steps $(\Delta x, \Delta y)$ to the current state.

- **Transition Rule:** If a successor state is better (higher $f(x, y)$ value), the transition is accepted. If worse, it's accepted with probability $P(\text{accept}) = e^{\frac{\Delta E}{T}}$, where ΔE is the change in $f(x, y)$ and T is the current temperature.
- **Cooling Schedule:** A linear cooling schedule was proposed and implemented, $T(t) = \max(0.0, T_{\text{initial}} - \text{cooling_rate} \times t)$, where T decreases as iteration t increases.

3) *Example of a Single SA Step (Hypothetical):* Consider a current state $(x_c, y_c) = (0, 0)$ with objective value $f_c = 2.0$, at temperature $T = 10.0$. A randomly chosen neighboring state (x_n, y_n) is worse, with $f_n = 1.9$. (The objective function is $f(x, y) = \sin(x/8) + \cos(y/4) - \sin(xy/16) + \cos(x^2/16) + \sin(y^2/8)$).

The decision to move to this worse state (a "downhill" move in maximization, helping to escape local optima) is probabilistic:

- Calculate change in value: $\Delta E = f_n - f_c = 1.9 - 2.0 = -0.1$.
- Determine acceptance probability: $P = e^{\Delta E/T} = e^{-0.1/10.0} = e^{-0.01} \approx 0.99005$.
- Accept the move if a random number $r \in [0, 1)$ is less than P . (E.g., if $r = 0.5$, accept; if $r = 0.995$, reject).

4) *System Implement:* Object-Oriented Programming (OOP).

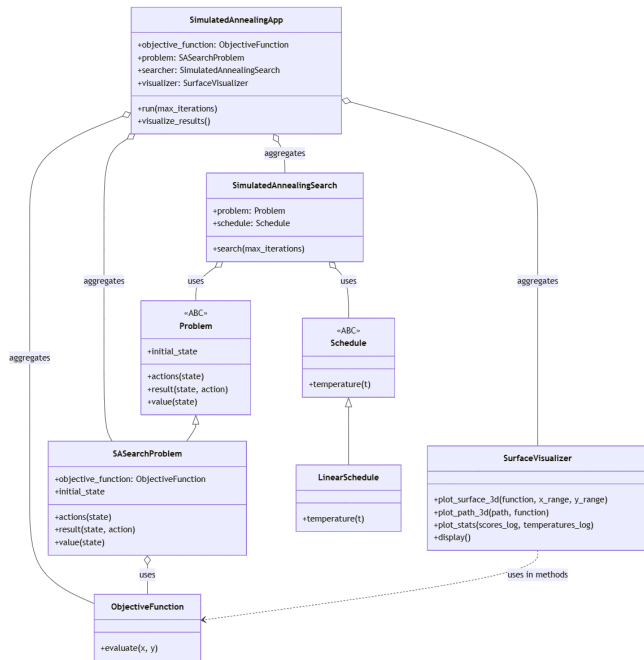


Fig. 1. Task1 - Simple Class Diagram

Algorithm 1 Simulated Annealing

```

1: procedure SIMULATED-ANNEALING(problem, schedule)
2:   current  $\leftarrow$  INITIAL-STATE(problem)
3:   for  $t \leftarrow 1$  to MAX_ITERATIONS do
4:      $T \leftarrow$  SCHEDULE( $t$ )
5:     if  $T = 0$  then
6:       return current
7:     end if
8:     next  $\leftarrow$  RANDOM-SUCCESSOR(current, problem)
9:      $\Delta E \leftarrow$  VALUE(next, problem) - VALUE(current, problem)
10:    if  $\Delta E > 0$  then
11:      current  $\leftarrow$  next
12:    else
13:      if RANDOM()  $< e^{\Delta E/T}$  then
14:        current  $\leftarrow$  next
15:      end if
16:    end if
17:  end for
18:  return current
19: end procedure
20: Linear-Schedule(current_temp, cooling_rate): return
    max(0.0, current_temp - cooling_rate)
  
```

B. Task 2: 9x9 Tic-Tac-Toe with Heuristic Alpha-Beta Search

1) *Issue:* The goal is to create a 9×9 console Tic-Tac-Toe game where a human plays an AI. Victory requires four pieces in a row (horizontally, vertically, or diagonally). The main challenge is designing an intelligent AI using heuristic alpha-beta search that performs well on this larger board.

2) *Approach and Formulate:* The AI uses **heuristic alpha-beta search**, an optimized minimax algorithm for game tree exploration.

- **Game Tree & Pruning:** The game is a tree of states; alpha-beta pruning eliminates branches that won't alter the outcome.
- **Players:** Human ('X') is MAX, AI ('O') is MIN.
- **Depth Limit (L):** Search is restricted to a depth L due to the large board size.
- **Heuristic Function $h(s)$:** At depth L or non-terminal states, $h(s)$ estimates state utility. The WeightedLineHeuristic scores potential 4-in-a-row lines:

- Lines with only human pieces (X_{count}) get $10^{X_{\text{count}}}$.
- Lines with only AI pieces (O_{count}) get $-10^{O_{\text{count}}}$.
- Blocked or empty lines score 0.

Rationality: This prioritizes forming/blocking winning lines, with exponential scores for longer unblocked sequences. Terminal wins get dominating scores ($\pm 10^{\text{win_length}+2}$).

3) *Example: Heuristic Scores for Line Segments:* Consider a win length of 4, where Human is 'X' (MAX player) and AI is 'O' (MIN player). The symbol '_' denotes an empty cell. The heuristic evaluates potential 4-cell line segments, priori-

tizing lines with more pieces and no opponent obstruction by assigning exponentially increasing scores:

- X X _ _ (Composition: 2 'X', 0 'O', 2 empty). This line is unblocked for Human (MAX). Score: $10^2 = 100$.
- O O O _ (Composition: 3 'O', 0 'X', 1 empty). This line is unblocked for AI (MIN). Score: $-10^3 = -1000$.
- X O _ _ (Composition: 1 'X', 1 'O', 2 empty). This line is contested by both players (blocked), offering no immediate advantage to either. Score: 0.

The Alpha-Beta search algorithm utilizes these heuristic scores when it reaches its predefined search depth. Positive scores indicate an advantage for 'X', while negative scores favor 'O'. The AI (as MIN player) seeks moves that lead to states with the lowest scores.

4) *System Implement:* Object-Oriented Programming (OOP).

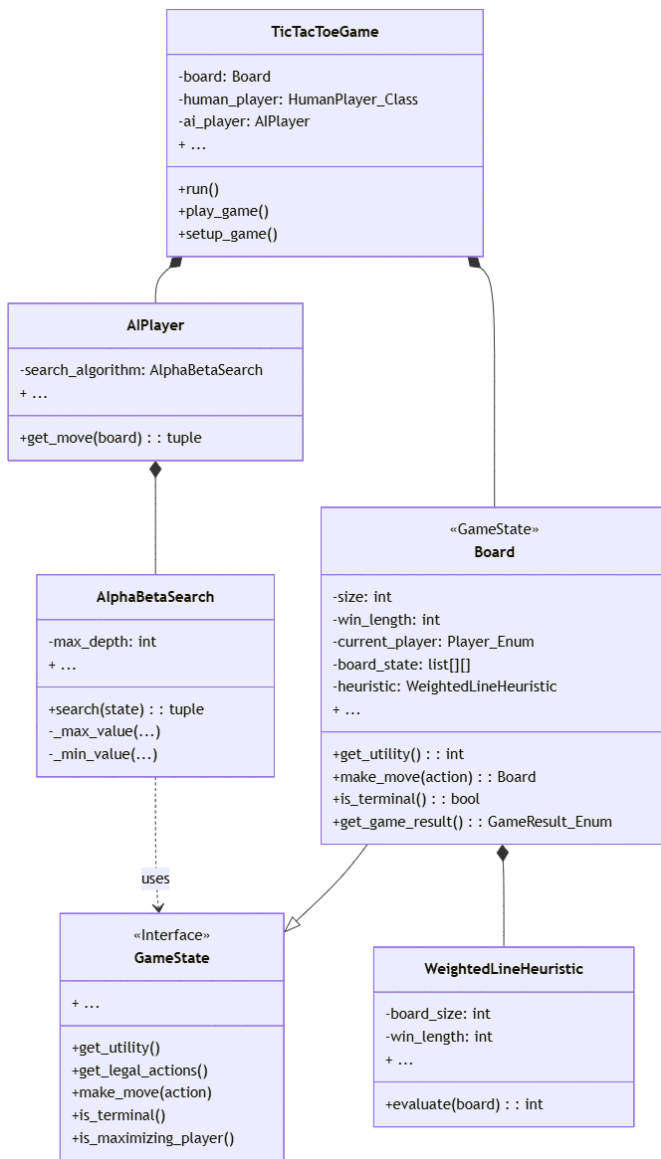


Fig. 2. Task2 - Simple Class Diagram

Algorithm 2 Alpha-Beta Search (Further Separated Calls)

```

1: function ALPHA-BETA-SEARCH(state)
2:    $v_{utility} \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$ 
3:   return action  $a$  in ACTIONS(state) such that its evaluated utility is  $v_{utility}$ 
4: end function

5: function MAX-VALUE(state,  $\alpha$ ,  $\beta$ )
6:   if TERMINAL-TEST(state) then
7:     return UTILITY(state)
8:   end if
9:    $v \leftarrow -\infty$ 
10:  for each  $a$  in ACTIONS(state) do
11:     $next\_state \leftarrow \text{RESULT}(\text{state}, a)$ 
12:     $child\_utility \leftarrow \text{MIN-VALUE}(next\_state, \alpha, \beta)$ 
13:     $v \leftarrow \text{MAX}(v, child\_utility)$ 
14:    if  $v \geq \beta$  then
15:      return  $v$ 
16:    end if
17:     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
18:  end for
19:  return  $v$ 
20: end function

21: function MIN-VALUE(state,  $\alpha$ ,  $\beta$ )
22:   if TERMINAL-TEST(state) then
23:     return UTILITY(state)
24:   end if
25:    $v \leftarrow +\infty$ 
26:   for each  $a$  in ACTIONS(state) do
27:     $next\_state \leftarrow \text{RESULT}(\text{state}, a)$ 
28:     $child\_utility \leftarrow \text{MAX-VALUE}(next\_state, \alpha, \beta)$ 
29:     $v \leftarrow \text{MIN}(v, child\_utility)$ 
30:    if  $v \leq \alpha$  then
31:      return  $v$ 
32:    end if
33:     $\beta \leftarrow \text{MIN}(\beta, v)$ 
34:  end for
35:  return  $v$ 
36: end function
  
```

C. Task 3: Constraint Satisfaction Problems with Propositional Logic

1) *Issue:* We color an $m \times n$ grid with Red or Green, adhering to numerical constraints on specific cells. Each cell containing a number k dictates that exactly k of its 9 "adjacent" neighbors (which includes the cell itself plus the eight surrounding cells) must be colored Green. By following requires:

- Find the general rules to generate the constraints for each cell in the matrix.
- Using the Glucose3 solver to solving the problem.

2) *Approach And Formulate:* We can approach it using a SAT solver due to the following reasons:

- Each cell in the grid can be modeled as a Boolean variable.

- All variables have the same binary domain: Green, Red, where True may represent Green and False represents Red.

What remains is defining the constraint set. The main rule is: "If a cell has a value k , then exactly k of its neighboring cells must be Green, and the rest must be Red." To enforce this rule, we encode it in Conjunctive Normal Form (CNF) using propositional logic. CNF form statement is True if and only if:

- A clause is satisfied if at least one literal is true.
- A formula is satisfied if all clauses are true.

To ensure that exactly k neighbors are Green, we must eliminate invalid assignments:

- Those with less than k Green neighbors.
- Those with more than k Green neighbors.

This is done by constructing blocking clauses:

- For each subset of $k+1$ neighbors, we add a clause that ensures that not all of them can be Green.
- For each subset of neighbors $n - k + 1$, we add a clause to ensure that not all of them can be Red.

This encoding guarantees that only assignments with exactly k Green neighbors are allowed, satisfying the constraints of the problem. We then use the SAT algorithm provided by the Glucose3 solver to find a valid assignment that satisfies all constraints of the problem.

3) *Example: CNF for "Exactly k out of n ":* We aim to build CNF clauses for the constraint that exactly $k = 3$ out of $n = 4$ variables ($\{x_1, x_2, x_3, x_4\}$) are True. Each x_i is Boolean (True = Green, False = Red). This is achieved by ensuring "at least 3 are True" AND "at most 3 are True".

- **At Most $k = 3$ True:** To ensure no more than 3 variables are True, we forbid any $k+1 = 4$ variables from all being True.
 - Consider all subsets of variables of size $k+1 = 4$. For $n = 4, k = 3$, there's one such subset: $\{x_1, x_2, x_3, x_4\}$.
 - For this subset, the clause is: $(\neg x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4)$. This means that not all 4 variables can be True simultaneously.
- **At Least $k = 3$ True:** To ensure at least 3 variables are True, we forbid any $n - k + 1 = (4 - 3 + 1) = 2$ variables from both being False.
 - Consider all subsets of variables of size $n - k + 1 = 2$. There are $\binom{4}{2} = 6$ such subsets.
 - For each subset (e.g., $\{x_1, x_2\}$), add a clause requiring at least one variable in it to be True (i.e., they cannot both be False).
 - Clauses:

$$\begin{aligned} (x_1 \vee x_2), & \quad (x_1 \vee x_3), & \quad (x_1 \vee x_4), \\ (x_2 \vee x_3), & \quad (x_2 \vee x_4), & \quad (x_3 \vee x_4) \end{aligned}$$

Combined CNF Clauses for "Exactly 3 out of 4 True":
The conjunction (AND) of all generated clauses enforces the constraint:

$$\begin{aligned} & (\neg x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4) \\ & \wedge (x_1 \vee x_2) \wedge (x_1 \vee x_3) \wedge (x_1 \vee x_4) \\ & \wedge (x_2 \vee x_3) \wedge (x_2 \vee x_4) \wedge (x_3 \vee x_4) \end{aligned}$$

4) *System Implement:* Object-Oriented Programming (OOP):

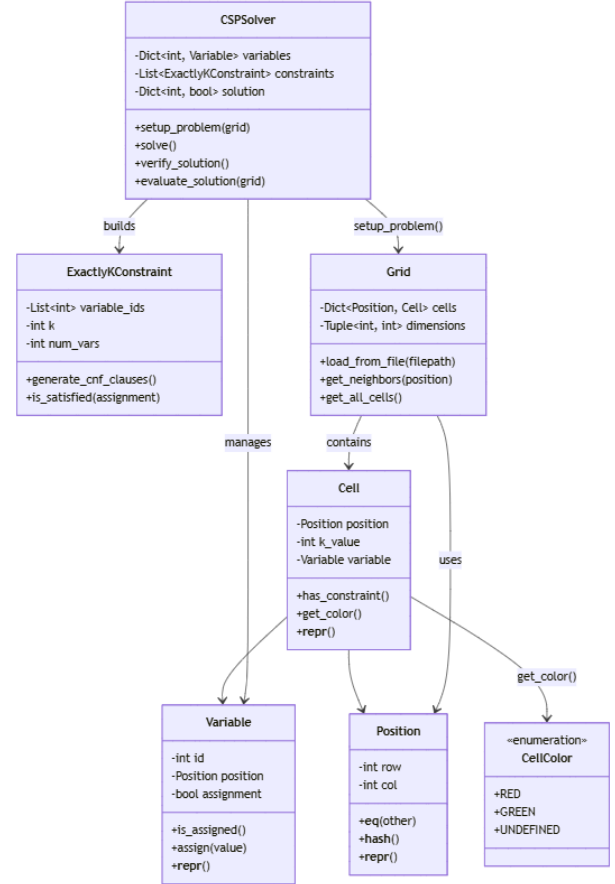


Fig. 3. Task3 - Simple Class Diagram

D. Task 4: Naïve Bayesian Classifier

1) *Issue:* Manually building a Naïve Bayesian classifier to predict student course outcomes (Rank: Pass/Fail) from nine quiz scores (Q1-Q9) involves addressing:

- The need for discretization of continuous quiz scores.
- Potential missing values in the input data during prediction.
- Ensuring that missing values in the training dataset (if not handled) do not lead to incorrect probability computations.

2) *Approach and Formulation:*

- **Data Discretization:** Quiz scores are discretized using an equal-width binning method based on the following predetermined bins and categories:

Bins	Categories
[0.0, 2.5]	Very Low (VL)
[2.6, 5.0]	Low (L)
[5.1, 7.5]	Medium (M)
[7.6, 10]	High (H)

• **Naïve Bayesian Classifier Implementation:**

- **Training Phase:** Probabilities are calculated using Laplace smoothing (add-1) to prevent zero-probability issues.

* **Class Prior Probability $P(C_i)$:**

$$P(C_i) = \frac{\text{count}(C_i) + 1}{\text{Total Samples } (|D|) + k}$$

Where: $\text{count}(C_i)$ is samples in class C_i ; $|D|$ is total samples; k is the number of unique classes.

* **Attribute Likelihood $P(X_j = v_j|C_i)$:**

$$P(X_j = v_j|C_i) = \frac{\text{count}(X_j = v_j \wedge C_i) + 1}{\text{count}(C_i) + N_j}$$

Where: $\text{count}(X_j = v_j \wedge C_i)$ is samples in class C_i with attribute X_j having value v_j ; N_j is the number of unique values for attribute X_j .

* **Instance Likelihood $P(X|C_i)$:** Based on the "naïve" assumption of conditional independence among attributes $(X_1, \dots, X_n)$:

$$P(X|C_i) = \prod_{j=1}^n P(X_j = x_j|C_i)$$

Where $X = (x_1, \dots, x_n)$ is the instance with n attributes.

– **Prediction Phase (Testing):**

* **Predicting for an Instance X :** The class C_{NB} is predicted by maximizing the posterior probability, which is proportional to the prior multiplied by the instance likelihood:

$$C_{NB} = \arg \max_{C_i} \left(P(C_i) \prod_{j=1}^n P(X_j = x_j|C_i) \right)$$

(The $P(X)$ term is omitted as it's constant across classes).

* **Evaluating Model Accuracy:** Predictions are made for all dataset entries to assess the accuracy against the true labels.

3) **Example: Example for predicting an instance**

Given a discretized instance with quiz scores (Q1 to Q9):

Q1=M, Q2=M, Q3=M, Q4=VL, Q5=VL, Q6=H, Q7=H, Q8=M, Q9=H.

1. Calculate Proportional Posterior Scores (S_{Class}): Using prior probabilities ($P(\text{Rank}=P) \approx 178/269$, $P(\text{Rank}=F) \approx 91/269$) and likelihoods $P(Q_i|\text{Rank}=Class)$ from the trained model. Let $S_{\text{Class}} = P(\text{Rank}=Class) \times \prod_{i=1}^9 P(Q_i|\text{Rank}=Class)$.

- For Rank = P: $S_P \approx 2.088 \times 10^{-5}$
- For Rank = F: $S_F \approx 8.05 \times 10^{-6}$

Let the sum of scores be $S_{\text{total}} = S_P + S_F \approx 2.088 \times 10^{-5} + 8.05 \times 10^{-6} \approx 2.893 \times 10^{-5}$.

2. Normalize Scores and Predict Class: The posterior probabilities are normalized:

$$P(\text{Rank}=P|X) = \frac{S_P}{S_{\text{total}}} = \frac{2.088 \times 10^{-5}}{2.893 \times 10^{-5}} \times 100\% \approx 72.2\%$$

$$P(\text{Rank}=F|X) = \frac{S_F}{S_{\text{total}}} = \frac{8.05 \times 10^{-6}}{2.893 \times 10^{-5}} \times 100\% \approx 27.8\%$$

Since $P(\text{Rank}=P|X) > P(\text{Rank}=F|X)$, the predicted class is:

⇒ Predicted class: Rank = P

4) **System Implement:** Object-Oriented Programming (OOP).

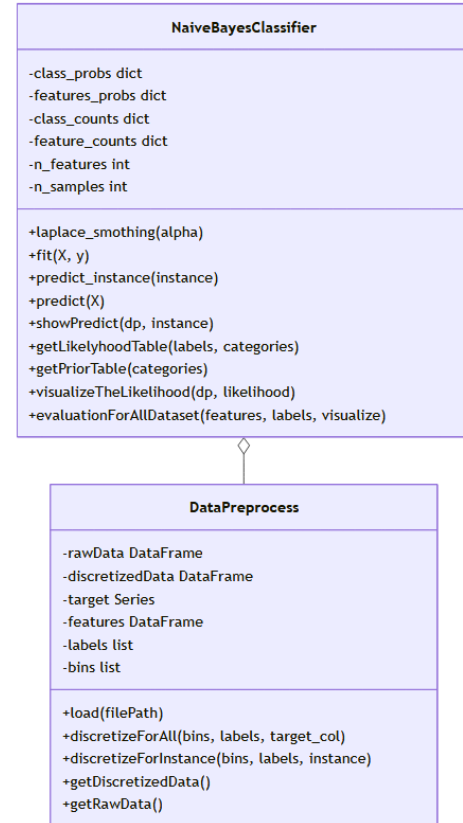


Fig. 4. Task4 - Simple Class Diagram

III. EXPERIMENTAL RESULTS AND EVALUATION

A. Simulated Annealing

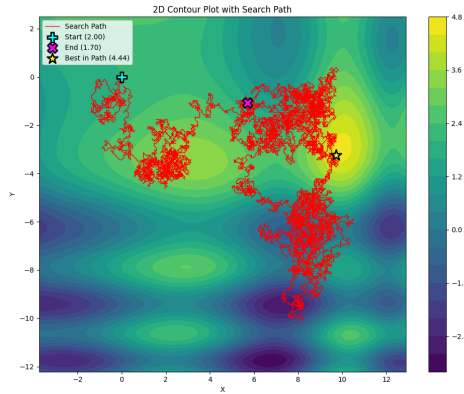


Fig. 5. Task1 - 2D Contour Plot with Search Path

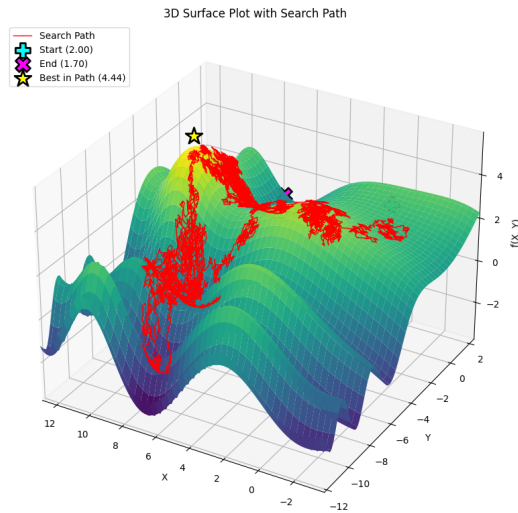


Fig. 6. Task1 - 3D Surface Plot with Search Path

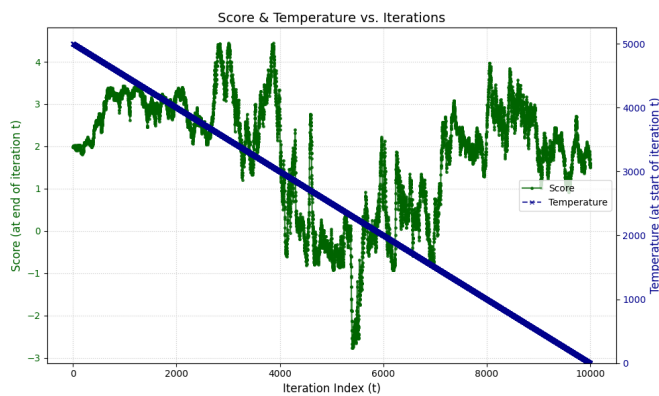


Fig. 7. Task1 - Score-Temperature vs. Iterations

Initial state: (0.0, 0.0), Value: 2.0000
Best solution found by SA: (9.7193, -3.2398)
Value at best SA solution: 4.4447

Fig. 8. Task1 - Results

B. Evaluation of Tic-Tac-Toe AI

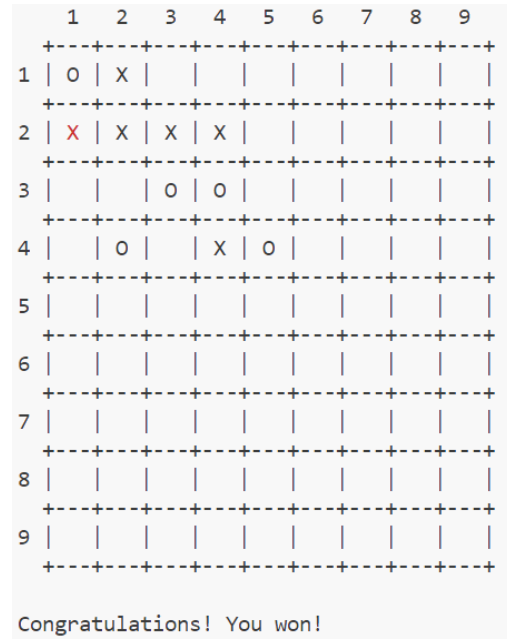


Fig. 9. Task2 - Player win

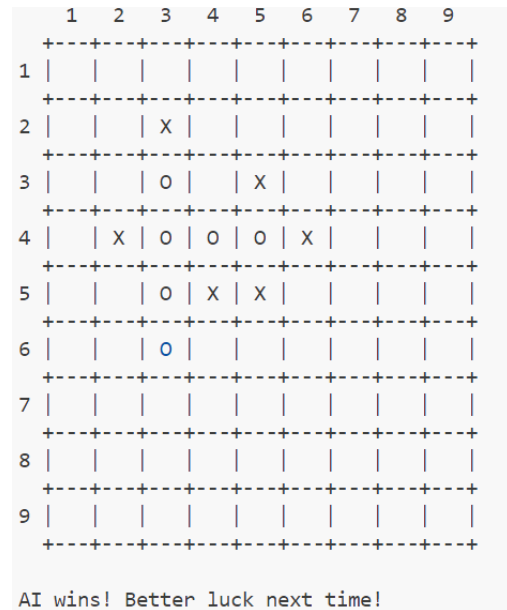


Fig. 10. Task2 - AI win

C. SAT Solver Results for Matrix Coloring

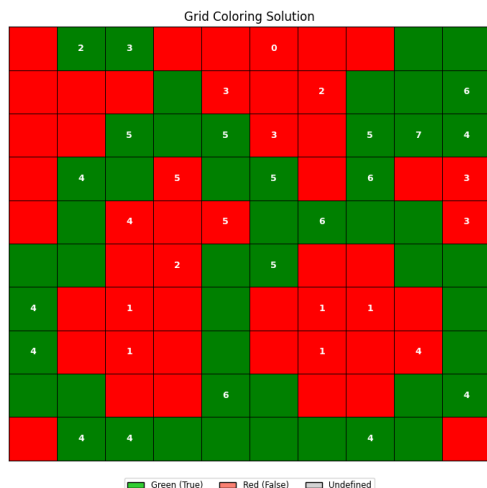


Fig. 11. Task3 - Results Matrix (SAT)

```

=== AI Grid Coloring CSP Solver ===
Successfully loaded grid: 10x10
Setup complete: 100 variables, 36 constraints
Solution found!
Solution verified successfully!

=== Solution Evaluation ===
Grid size: 10x10
Variables: 100
Constraints: 36
CellPosition(0, 1): k=2, actual=2 - CORRECT
CellPosition(0, 2): k=3, actual=3 - CORRECT
...
Accuracy: 100.0% (36/36)

```

Fig. 12. Task3 - Evaluation of result matrix (SAT)

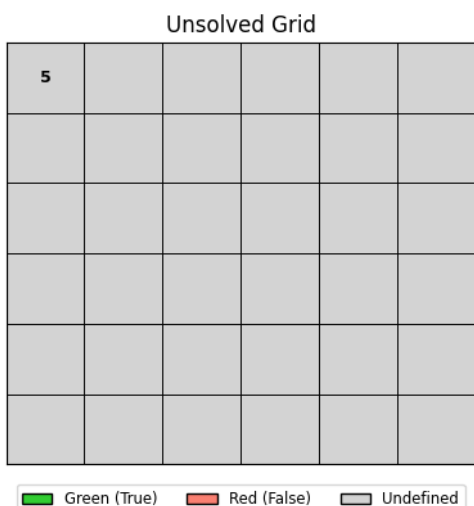


Fig. 13. Task3 - Results Matrix (UNSAT)

```

=== AI Grid Coloring CSP Solver ===
Successfully loaded grid: 6x6
Setup complete: 36 variables, 1 constraints
No solution exists (UNSATISFIABLE)

```

Fig. 14. Task3 - Evaluation of result matrix (UNSAT)

D. Naïve Bayes Classifier Accuracy

Attribute	Rank = P	Rank = F					
0	Q1=VL	96/181	71/94	19	Q5=H	11/181	3/94
1	Q1=L	74/181	19/94	20	Q6=VL	17/181	28/94
2	Q1=M	10/181	3/94	21	Q6=L	26/181	27/94
3	Q1=H	1/181	1/94	22	Q6=M	43/181	18/94
4	Q2=VL	26/181	31/94	23	Q6=H	95/181	21/94
5	Q2=L	23/181	11/94	24	Q7=VL	60/181	61/94
6	Q2=M	55/181	25/94	25	Q7=L	61/181	21/94
7	Q2=H	77/181	27/94	26	Q7=M	51/181	8/94
8	Q3=VL	27/181	30/94	27	Q7=H	9/181	4/94
9	Q3=L	69/181	41/94	28	Q8=VL	31/181	33/94
10	Q3=M	50/181	16/94	29	Q8=L	30/181	16/94
11	Q3=H	35/181	7/94	30	Q8=M	51/181	24/94
12	Q4=VL	91/181	71/94	31	Q8=H	69/181	21/94
13	Q4=L	42/181	8/94	32	Q9=VL	18/181	29/94
14	Q4=M	25/181	4/94	33	Q9=L	18/181	19/94
15	Q4=H	23/181	11/94	34	Q9=M	65/181	33/94
16	Q5=VL	39/181	40/94	35	Q9=H	80/181	13/94
17	Q5=L	90/181	36/94				
18	Q5=M	41/181	15/94				

Fig. 15. Task 4 - Likelihood table

Rank = P Rank = F

178 / 269 91 / 269

Fig. 16. Task4 - Class Prior Probability

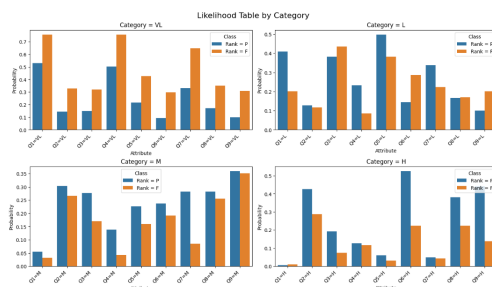


Fig. 17. Task4 - Prediction Results (Detail)

Predict for instance 1:			Predict for instance 3:		
Non-discretized			Discretized		
Q1	0.0	VL	Q1	2.0	VL
Q2	0.0	VL	Q2	9.0	H
Q3	0.0	VL	Q3	7.0	M
Q4	0.0	VL	Q4	8.0	H
Q5	2.0	VL	Q5	6.0	M
Q6	6.0	M	Q6	1.0	VL
Q7	0.0	VL	Q7	5.0	L
Q8	6.4	M	Q8	6.4	M
Q9	6.4	M	Q9	9.0	H
Predicted Class: F, Confidence: 93.6829665396088			Predicted Class: P, Confidence: 89.51541821972977		
Predict for instance 2:			Predict for instance 4:		
Non-discretized			Discretized		
Q1	1	VL	Q1	5	L
Q2	1	VL	Q2	9	H
Q3	4	L	Q3	5	L
Q4	5	L	Q4	8	H
Q5	6	M	Q5	6	M
Q6	7	M	Q6	5	L
Q7	9	H	Q7	5	L
Q8	7	M	Q8	5	L
Q9	8	H	Q9	9	H
Predicted Class: P, Confidence: 91.18771557129841			Predicted Class: P, Confidence: 94.8809338822104		

Fig. 18. Instance predictions

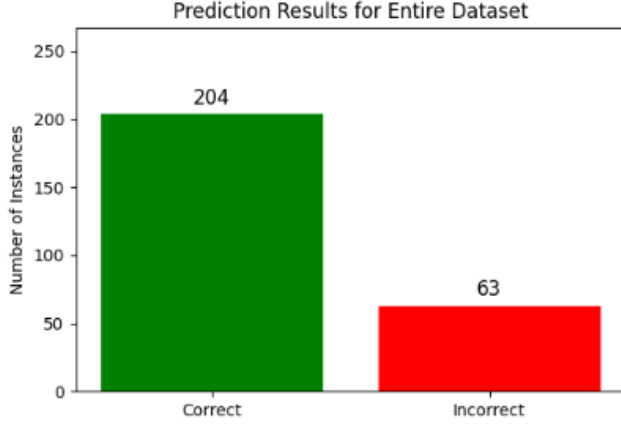


Fig. 19. Task4 - Overall Accuracy

IV. CONTRIBUTIONS

TABLE I
INDIVIDUAL CONTRIBUTIONS

Name	Assignments	Percentage (%)
Nguyen Tran Hoang Nhan	Task 1	100
Phan Thanh Dat	Task 2	100
Nguyen Nhut Khang	Task 3	100
Mai Hoang Thai	Task 4	100
Nguyen Vo Thanh Tu	Task 5 (Report)	100

V. SELF-EVALUATION

TABLE II
SELF-EVALUATION OF TASK COMPLETION AND ESTIMATED SCORES

Task	Evaluation Criteria	Estimated Score
Task 1 (2.0 pts)	OOP Structure	2.0/2.0
	Correct Approach (SA)	
	Correct Result & Visualization	
Task 2 (3.0 pts)	OOP Structure	3.0/3.0
	Correct Approach (Alpha-Beta)	
	Correct Result & Heuristic Rationale	
Task 3 (3.0 pts)	OOP Structure	3.0/3.0
	Correct Approach (SAT)	
	Correct Result & Visualization	
Task 4 (1.0 pt)	OOP Structure	1.0/1.0
	Correct Approach (Naive Bayes)	
	Correct Result & Accuracy	
Task 5 (1.0 pt)	Report Format (IEEE)	1.0/1.0
	Comprehensive Content	
Total Estimated Score		10.0/10.0

VI. CONCLUSION

This project culminated in the successful application and rigorous evaluation of diverse Artificial Intelligence paradigms across four distinct computational challenges, spanning optimization, strategic game playing, logical constraint satisfaction, and probabilistic classification. Simulated Annealing proved effective in navigating a complex 3D search space to identify high-value regions. A competitive AI for 9×9 Tic-Tac-Toe was engineered using heuristic Alpha-Beta search, demonstrating strategic depth. The challenge of a matrix coloring puzzle was adeptly resolved by translating its constraints into propositional logic and leveraging a SAT solver. Furthermore, a Naïve Bayesian classifier was meticulously implemented from first principles, including a tailored data discretization strategy, to accurately predict student academic ranks. An Object-Oriented design philosophy underpinned all development efforts, contributing to modular and robust solutions. This endeavor not only fulfilled all specified requirements to a high standard but also significantly enhanced our practical understanding and proficiency in these core AI techniques and their versatile application to real-world problems.

REFERENCES

- [1] Stuart Russell, Peter Norvig, Artificial Intelligence: A Modern Approach, 3rd edition, Published by Pearson, USA, 2005.
- [2] Tom Mitchell, Machine Learning, McGraw-Hill Press, USA, 1997.
- [3] Kevin P. Murphy, Machine Learning: A Probabilistic Perspective (Adaptive Computation and Machine Learning), 1st edition, Published by MIT Press, USA, 2012.
- [4] Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, USA, 2016.
- [5] Dinh Manh Tuong, Giao trinh Tri Tue Nhan Tao, NXB Dai hoc Quoc gia Ha Noi, Ha Noi, 2008.